

ui.number 全面详细阐述

`ui.number` 是 NiceGUI 框架中基于 Quasar QInput 组件实现的数字输入组件，专注于整数 / 浮点数输入场景，支持数值范围限制、步长调整、格式化显示、表单验证等核心功能，适用于数量录入、参数配置、数值筛选等需要精确数字输入的场景。以下从核心特性、使用方法、参数配置、API 详情等方面展开全面说明。

一、核心特性

- 精准数字输入**：仅允许输入数字（整数 / 浮点数），自动过滤非数字字符，支持正负号、小数点输入。
- 灵活范围与步长控制**：通过 `min/max` 限制数值范围，`step` 控制增减步长（支持整数、小数步长）。
- 便捷操作体验**：内置增减按钮（可通过 `controls` 参数控制显示 / 隐藏），支持鼠标滚轮调整数值，提升操作效率。
- 格式化显示**：支持通过 `format` 参数自定义数值显示格式（如千分位分隔、固定小数位数），优化数值可读性。
- 完善验证机制**：支持范围验证、自定义函数验证（含异步验证），可手动触发验证并获取错误信息。
- 组件联动能力**：支持与滑块、标签、开关等组件进行值绑定、启用状态绑定、可见性绑定，实现复杂交互逻辑。
- 样式可定制**：支持 Quasar 原生属性（如圆角、边框、紧凑模式），可通过 `classes/style` 自定义组件样式。

二、基础使用方法

1. 最简示例：基础数字输入

快速创建数字输入框，设置标签、默认值、范围限制和步长，实时监听数值变化：

```
from nicegui import ui

# 标签用于实时显示输入结果
result = ui.label('当前值: 25')

# 基础数字输入框
ui.number(
    label='数量选择',
    value=25, # 初始值
    min=0,   # 最小值
    max=100, # 最大值
    step=5,  # 增减步长
    on_change=lambda e: result.set_text(f'当前值: {e.value}') # 实时响应变化
)

ui.run()
```

2. 浮点数输入（小数步长）

通过 `step` 设置小数步长，支持高精度浮点数输入：

```
from nicegui import ui

# 浮点数输入（步长 0.1，范围 0-10）
ui.number(
    label='精度调整',
    value=3.5,
    min=0,
    max=10,
    step=0.1, # 小数步长
    format='.2f' # 格式化显示：保留 2 位小数
)

# 高精度浮点数（步长 0.001）
ui.number(
    label='高精度输入',
    value=1.234,
    min=0,
    max=5,
    step=0.001,
    format='.3f' # 保留 3 位小数
)

ui.run()
```

3. 控制增减按钮显示

通过 `controls` 参数控制内置增减按钮的显示 / 隐藏（默认显示）：

```
from nicegui import ui

# 显示增减按钮（默认）
ui.number(label='显示控制按钮', value=10, min=0, max=100, step=2)

# 隐藏增减按钮（仅通过输入框或滚轮调整）
ui.number(
    label='隐藏控制按钮',
    value=50,
    min=0,
    max=100,
    controls=False # 隐藏增减按钮
)

ui.run()
```

4. 数值格式化显示

通过 `format` 参数自定义数值格式，支持 Python 字符串格式化语法（如千分位、固定小数位）：

```
from nicegui import ui

# 千分位分隔（整数）
ui.number(
    label='大额数值',
    value=12345,
    min=0,
    max=100000,
    step=1000,
    format=',d' # 千分位分隔（如 12,345）
)

# 固定小数位 + 千分位（浮点数）
ui.number(
    label='金额输入',
    value=1234.56,
    min=0,
    max=10000,
    step=0.01,
    format=',.2f' # 千分位分隔 + 保留 2 位小数（如 1,234.56）
)

# 科学计数法（高精度数值）
ui.number(
    label='科学计数法',
    value=1234567.89,
    format='.2e' # 科学计数法（如 1.23e+06）
)

ui.run()
```

5. 输入验证（范围 + 自定义规则）

支持内置范围验证（`min/max` 自动生效）和自定义验证函数（含异步验证）：

```
from nicegui import ui
import asyncio

# 基础范围验证（min/max 自动验证）
ui.number(
    label='年龄输入',
    value=20,
    min=18, # 最小值限制（小于 18 触发错误）
    max=60, # 最大值限制（大于 60 触发错误）
    label='年龄（18-60 岁）'
)

# 自定义函数验证（偶数验证）
```

```

ui.number(
    label='偶数输入',
    value=4,
    min=0,
    max=100,
    step=2,
    validation=lambda value: '请输入偶数' if value % 2 != 0 else None
)

# 异步验证（模拟后端校验）
async def async_validate(value):
    await asyncio.sleep(0.5) # 模拟接口请求延迟
    return '数值已被占用' if value == 50 else None

ui.number(
    label='异步验证',
    value=30,
    min=0,
    max=100,
    validation=async_validate
)

ui.run()

```

三、关键参数说明

- `label`：类型为 `str | None`，用于说明输入用途的组件显示标签，无默认值。
- `value`：类型为 `int | float | None`，支持整数或浮点数的初始数值，默认值为 `None`。
- `min`：类型为 `int | float | None`，允许输入的最小值，低于该值会触发验证错误，默认值为 `None`（无下限）。
- `max`：类型为 `int | float | None`，允许输入的最大值，高于该值会触发验证错误，默认值为 `None`（无上限）。
- `step`：类型为 `int | float`，数值增减的步长，支持整数或小数（如 `0.1`、`0.001`），默认值为 `1`。
- `precision`：类型为 `Optional[int]`，允许的小数位数（默认无限制），负值表示小数点前的位数限制，数值会在失去焦点、相关参数变化或调用 `sanitize()` 时自动取整。
- `prefix`：类型为 `str | None`，用于在显示数值前添加的前缀内容，无默认值。
- `suffix`：类型为 `str | None`，用于在显示数值后添加的后缀内容，无默认值。
- `format`：类型为 `str | None`，类似 `"%.2f"` 的数值显示格式化字符串，可自定义数值展示形式，无默认值。
- `on_change`：类型为 `Callable[[ValueChangeEventArguments], Any] | Callable[[], Any]`，数值变化时触发的回调函数，事件对象 `e` 包含 `value` 属性（当前数值），无默认值。
- `validation`：类型为 `Union[Callable[[Any], Union[str, NoneType, Awaitable[Optional[str]]]], dict[str, Callable[[Any], bool]], NoneType]`，输入验证规则。支持字典（错误提示对应验证函数）或自定义函数（返回错误提示或 `None`，支持异步），默认值为 `None`（无额外验证）。

四、高级功能

1. 组件双向绑定（数字输入 ↔ 滑块）

将 `ui.number` 与 `ui.slider` 绑定，实现数值同步调整（适用于参数可视化配置）：

```
from nicegui import ui

# 数字输入框与滑块双向绑定
number_input = ui.number(label='数值调整', value=50, min=0, max=100, step=1)
slider = ui.slider(value=50, min=0, max=100, step=1)

# 双向绑定：一方变化，另一方同步更新
number_input.bind_value(slider)
slider.bind_value(number_input)

# 实时显示当前值
ui.label().bind_text_from(number_input, 'value', forward=lambda x: f'当前值: {x}')

ui.run()
```

2. 动态修改属性（范围、步长、格式）

通过 `set_min()`、`set_step()`、`set_format()` 等方法动态更新组件属性：

```
from nicegui import ui

number_input = ui.number(
    label='动态配置',
    value=10,
    min=0,
    max=100,
    step=5,
    format='d'
)

# 动态修改范围
ui.button('扩大范围 (0-200)', on_click=lambda: (
    number_input.set_min(0),
    number_input.set_max(200)
))

# 动态修改步长
ui.button('步长改为 10', on_click=lambda: number_input.set_step(10))

# 动态修改格式化
ui.button('添加千分位', on_click=lambda: number_input.set_format=',d'))

ui.run()
```

3. 禁用自动验证与手动触发验证

通过 `without_auto_validation()` 禁用实时自动验证，仅手动触发验证（适用于表单提交场景）：

```
from nicegui import ui

# 禁用自动验证
number_input = ui.number(
    label='手动验证',
    value=30,
    min=10,
    max=50,
    validation=lambda value: '数值必须是 10 的倍数' if value % 10 != 0 else None
).without_auto_validation()

# 手动触发验证按钮
def trigger_validation():
    is_valid = number_input.validate() # 手动触发验证
    error_msg = number_input.error or '验证通过'
    ui.notify(f'验证结果: {error_msg}')

ui.button('点击验证', on_click=trigger_validation)

ui.run()
```

4. 启用 / 禁用与显示 / 隐藏控制

通过 `set_enabled()`、`set_visibility()` 或绑定开关组件，控制数字输入框状态：

```
from nicegui import ui

number_input = ui.number(label='可控制输入框', value=50, min=0, max=100)

# 开关控制启用/禁用
enable_switch = ui.switch(label='启用输入', value=True)
enable_switch.bind_value_to(number_input, 'enabled')

# 开关控制显示/隐藏
show_switch = ui.switch(label='显示输入框', value=True)
show_switch.bind_value_to(number_input, 'visible')

ui.run()
```

5. 滚轮调整灵敏度控制（结合 Quasar 属性）

通过 `props` 参数设置 Quasar 原生属性 `wheel-step`，控制鼠标滚轮调整数值的灵敏度：

```
from nicegui import ui

# 低灵敏度（滚轮每滚动一次调整 1）
ui.number(
```

```
        label='低灵敏度滚轮',
        value=50,
        min=0,
        max=100,
        props='wheel-step=1' # 滚轮步长=1
    )

# 高灵敏度（滚轮每滚动一次调整 10）
ui.number(
    label='高灵敏度滚轮',
    value=50,
    min=0,
    max=100,
    props='wheel-step=10' # 滚轮步长=10
)

ui.run()
```

五、API 详情补充

1. 核心属性

属性名	类型	说明		
classes	str	组件 HTML 类名，支持 Tailwind/Quasar 类（如 w-64 宽度、border-2 边框）。		
client	Client	组件所属的客户端实例。		
enabled	BindableProperty	是否启用组件（可绑定，支持动态切换），默认值为 True。		
error	`str`	None`	验证失败后的错误信息（可手动设置或由验证规则自动生成）。	
format	`str`	None`	当前数值格式化字符串（可直接赋值修改，如 number_input.format = '.2f'）。	
html_id	`str`	None`	组件 HTML DOM ID（版本 2.16.0+）。	
label	BindableProperty	组件标签（可绑定，支持动态修改）。		
max	`int`	float	None`	当前最大值（可直接赋值修改，如 number_input.max = 200）。
min	`int`	float	None`	当前最小值（可直接赋值修改，如 number_input.min = 10）。
props	Props[Self]	组件 Quasar 原生属性（如 wheel-step、rounded、outlined）。		

属性名	类型	说明		
<code>step</code>	<code>`int</code>	<code>float`</code>	当前增减步长（可直接赋值修改，如 <code>number_input.step = 0.5</code> ）。	
<code>style</code>	<code>Style[Self]</code>	组件内联 CSS 样式（如 <code>height: 50px;</code> ）。		
<code>validation</code>	<code>`Callable</code>	<code>dict</code>	<code>None`</code>	验证规则（可动态修改，如 <code>number_input.validation = new_rule</code> ）。
<code>value</code>	<code>BindableProperty</code>	当前数值（可绑定，支持动态同步）。		
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（可绑定，支持动态切换）。		

2. 常用方法

方法名	说明	参数		
<code>bind_enabled(target)</code>	双向绑定组件启用状态到目标对象（如开关）。	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>enabled</code> ）。		
<code>bind_label(target)</code>	双向绑定组件标签到目标对象。	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>label</code> ）。		
<code>bind_value(target)</code>	双向绑定组件数值到目标对象（如滑块、标签）。	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>value</code> ）。		
<code>bind_visibility(target)</code>	双向绑定组件可见性到目标对象。	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>visible</code> ）。		
<code>set_enabled(bool)</code>	启用 / 禁用组件。	<code>bool</code> ：True 启用，False 禁用。		
<code>`set_format(format_str: str</code>	<code>None)`</code>	动态设置数值格式化字符串。	<code>format_str</code> ：格式化字符串（如 <code>','</code> 、 <code>.2f</code> ）或 <code>None</code> （取消格式化）。	
<code>`set_max(max_val: int</code>	<code>float</code>	<code>None)`</code>	动态设置最大值。	<code>max_val</code> ：新最大值（ <code>None</code> 表示无上限）。
<code>`set_min(min_val: int</code>	<code>float</code>	<code>None)`</code>	动态设置最小值。	<code>min_val</code> ：新最小值（ <code>None</code> 表示无下限）。

方法名	说明	参数		
<code>`set_step(step_val: int</code>	<code>float)`</code>	动态设置增减步长。	<code>step_val</code> : 新步长（支持整数或小数）。	
<code>`set_value(value: int</code>	<code>float</code>	<code>None)`</code>	动态设置组件数值。	<code>value</code> : 新数值（ <code>None</code> 表示清空）。
<code>set_visibility(bool)</code>	显示 / 隐藏组件。	<code>bool</code> : <code>True</code> 显示, <code>False</code> 隐藏。		
<code>validate(return_result=True)</code>	手动触发验证。	<code>return_result</code> : 是否返回验证结果（异步验证需设为 <code>False</code> ）；返回值: <code>True</code> 有效 / 错误信息字符串。		
<code>without_auto_validation()</code>	禁用实时自动验证（仅手动触发）。	无参数, 返回组件实例（支持链式调用）。		
<code>tooltip(text: str)</code>	为组件添加悬浮提示。	<code>text</code> : 提示文本内容。		
<code>update()</code>	强制更新组件状态到客户端。	无参数。		

3. 核心事件

事件名	说明	回调参数
<code>on_change</code>	数值实时变化时触发（如点击增减按钮、输入数字、滚轮调整）。	事件对象 <code>e</code> , 含 <code>value</code> 属性（当前数值）。
<code>keydown.enter</code>	按下回车键时触发（适用于表单提交场景）。	事件对象 <code>e</code> , <code>e.sender.value</code> 可获取当前数值。
<code>blur</code>	组件失去焦点时触发（适用于延迟确认输入）。	事件对象 <code>e</code> , <code>e.sender.value</code> 可获取当前数值。
<code>click</code>	点击组件时触发（可用于初始化操作）。	通用事件对象。

六、注意事项

- 数值类型兼容**: `value`、`min`、`max`、`step` 支持整数和浮点数混合使用（如 `min=0`、`max=10`、`step=0.5`），组件会自动处理类型转换。
- 格式化显示限制**: `format` 参数仅影响数值显示，不改变实际存储的数值类型（如 `format='.2f'` 显示 `3.14`，实际 `value` 仍为 `3.14` 浮点数）。
- 步长与精度问题**: 使用小数步长时（如 `0.1`），可能因浮点数精度导致数值累积误差（如 `0.1 * 3 = 0.30000000000000004`），建议通过 `format` 格式化显示或使用整数缩放（如 `step=1` 对应实际 `0.1`，存储时除以 10）。

4. **验证优先级**: `min/max` 基础验证优先级高于自定义验证, 若数值超出范围, 先触发范围错误, 再执行自定义验证。
5. **版本兼容性**: `html_id` 属性需 NiceGUI 2.16.0+, 异步验证需 2.7.0+, `strict` 参数绑定需 3.0.0+, 使用时需注意框架版本。
6. **滚轮调整限制**: 滚轮调整仅在组件获得焦点时生效, 若需全局生效, 可结合 `props='wheel-step=1'` 并确保组件聚焦。

七、应用场景

1. 数量录入: 表单中录入商品数量、订单数量等整数场景 (如电商购物车)。
2. 参数配置: 系统配置面板中调整数值参数 (如音量、亮度、精度阈值), 支持小数步长。
3. 数值筛选: 数据表格筛选时输入数值范围 (如价格区间、年龄范围)。
4. 表单验证: 需要严格限制数值范围或格式的场景 (如手机号、身份证号中的数字部分, 需结合自定义验证)。
5. 可视化联动: 与滑块、进度条等组件绑定, 实现数值可视化调整 (如数据可视化中的参数调节)。

`ui.number` 凭借精准的数字输入控制、灵活的配置选项和完善的验证机制, 成为 NiceGUI 中处理数字输入的核心组件, 适配从简单数量选择到复杂参数配置的各类场景, 兼顾易用性与专业性。